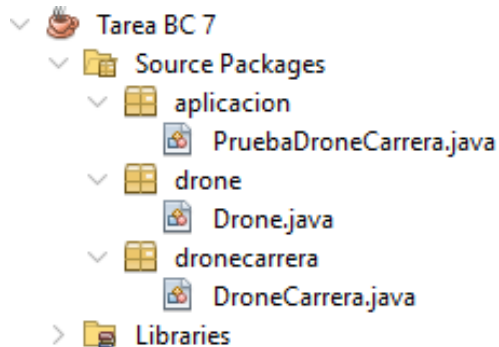




TAREA DEL BLOQUE DE CONTENIDOS 7

Cada una de las clases de los tres primeros ejercicios tiene que estar en un paquete diferente. Darles a los paquetes, y a las clases el mismo nombre que la estructura de la imagen:



1. Programar una clase de nombre Drone, que sirva como punto de partida para representar diferentes tipos de vehículos aéreos no tripulados. Un dron es una maravilla tecnológica capaz de desplazarse con autonomía, y esta clase modelará sus propiedades más fundamentales. Tendrá las siguientes propiedades y métodos:

Propiedades (privadas):

- ✓ peso: peso del dron en gramos.
- ✓ autonomia: tiempo de vuelo máximo en minutos.
- ✓ velocidadMaxima: velocidad máxima que puede alcanzar, en km/h.
- ✓ numMotores: número total de motores del dron.

Métodos de la clase (públicos):

- ✓ Constructor por defecto, que no realice ninguna acción.
- ✓ Constructor parametrizado, que reciba como argumentos los cuatro atributos y los inicialice.
- ✓ Métodos get y set para cada una de las propiedades.

Esta clase servirá como base sobre la que se construirán drones especializados, como los de carreras.

Solución:

```
package drone;
```

```
public class Drone {
```

```
    private int peso;  
    private int autonomia;  
    private int velocidadMaxima;  
    private int numMotores;
```



```
public Drone() {  
}  
  
public Drone(int peso, int autonomia, int velocidadeMaxima, int numMotores) {  
    this.peso = peso;  
    this.autonomia = autonomia;  
    this.velocidadeMaxima = velocidadeMaxima;  
    this.numMotores = numMotores;  
}  
  
public int getPeso() {  
    return peso;  
}  
  
public void setPeso(int peso) {  
    this.peso = peso;  
}  
  
public int getAutonomia() {  
    return autonomia;  
}  
  
public void setAutonomia(int autonomia) {  
    this.autonomia = autonomia;  
}  
  
public int getVelocidadeMaxima() {  
    return velocidadeMaxima;  
}  
  
public void setVelocidadeMaxima(int velocidadeMaxima) {  
    this.velocidadeMaxima = velocidadeMaxima;  
}  
  
public int getNumMotores() {  
    return numMotores;  
}  
  
public void setNumMotores(int numMotores) {  
    this.numMotores = numMotores;  
}  
}
```

2. Los drones de carreras representan un mundo apasionante donde la velocidad, la agilidad y la precisión son claves. Programar una clase, de nombre DroneCarrera, que perfeccionará el modelo para incluir características propias de este tipo de vehículos. Se trata de una extensión directa de la clase Drone, incorporando componentes técnicos más detallados.

Propiedades (privadas):



- ✓ potenciaMotores: array de 4 enteros, donde se almacena la potencia (en vatios) de cada motor.
- ✓ motorSeleccionado: número del motor activo (de 1 a 4).
- ✓ modoVuelo: puede tomar los valores "manual", "asistido" o "acrobático", que influyen en el comportamiento del dron.

Métodos de la clase (públicos):

- ✓ Constructor por defecto, que no realiza ninguna acción.
- ✓ Constructor con parámetros:
 - Llama al constructor de la clase padre para asignar peso, autonomia, velocidadMaxima y numMotores.
 - Asigna también las nuevas propiedades de esta clase (potenciaMotores, motorSeleccionado, modoVuelo).
- ✓ Métodos get y set para cada propiedad.
- ✓ Método visualizaDroneCarrera(), que imprime todos los datos técnicos del objeto con el siguiente formato:

Peso: 950 gramos
Autonomía: 18 minutos
Velocidad Máxima: 120 km/h
Número de motores: 4
Potencia de motores:

- Motor 1: 130 W
- Motor 2: 135 W
- Motor 3: 128 W
- Motor 4: 133 W

Motor seleccionado: 3
Modo de vuelo: acrobático
Velocidad estimada: 37.56 km/h

- ✓ Método velocidadEstimada(), que devuelve la velocidad estimada en km/h usando la siguiente lógica:
 1. Calcular el promedio de potencias activas.
 2. Aplicar un factor de eficiencia según el modo de vuelo:
 - "manual" → 0.75
 - "asistido" → 0.85
 - "acrobático" → 1.0
 - Cualquier otro → 0.70
 3. Aplicar una penalización por peso:
penalización = $1.0 - \min(0.001 \times \text{peso}, 0.4)$
 4. Fórmula final:
velocidad = potenciaPromedio × eficiencia × penalización × 0.1

Solución:

package dronecarrera;



```
import drone.Drone;

public class DroneCarrera extends Drone {

    private int[] potenciaMotores;
    private int motorSeleccionado;
    private String modoVuelo;

    public DroneCarrera() {
    }

    public DroneCarrera(int peso, int autonomia, int velocidadMaxima, int numMotores,
        int[] potenciaMotores, int motorSeleccionado, String modoVuelo) {
        super(peso, autonomía, velocidadMaxima, numMotores);
        this.potenciaMotores = potenciaMotores;
        this.motorSeleccionado = motorSeleccionado;
        this.modovuelo = modovuelo;
    }

    public int[] getPotenciaMotores() {
        return potenciaMotores;
    }

    public void setPotenciaMotores(int[] potenciaMotores) {
        this.potenciaMotores = potenciaMotores;
    }

    public int getMotorSeleccionado() {
        return motorSeleccionado;
    }

    public void setMotorSeleccionado(int motorSeleccionado) {
        this.motorSeleccionado = motorSeleccionado;
    }

    public String getModoVuelo() {
        return modoVuelo;
    }

    public void setModoVuelo(String modoVuelo) {
        this.modovuelo = modovuelo;
    }

    public double velocidadEstimada() {
        double promedio = 0;
        for (int potencia : potenciaMotores) {
            promedio += potencia;
        }
    }
}
```



```
}  
promedio /= potenciaMotores.length;  
  
double eficiencia;  
switch (modoVuelo.toLowerCase()) {  
    case "manual":  
        eficiencia = 0.75;  
        break;  
    case "asistido":  
        eficiencia = 0.85;  
        break;  
    case "acrobático":  
        eficiencia = 1.0;  
        break;  
    default:  
        eficiencia = 0.70;  
        break;  
}  
  
double penalizacion = 1.0 - Math.min(0.001 * getPeso(), 0.4);  
  
return promedio * eficiencia * penalizacion * 0.1;  
}  
  
public void visualizaDroneCarrera() {  
    System.out.println("Peso: " + getPeso() + " gramos");  
    System.out.println("Autonomía: " + getAutonomia() + " minutos");  
    System.out.println("Velocidad Máxima: " + getVelocidadMaxima() + " km/h");  
    System.out.println("Número de motores: " + getNumMotores());  
    System.out.println("Potencia de motores:");  
    for (int i = 0; i < potenciaMotores.length; i++) {  
        System.out.println(" - Motor " + (i + 1) + ": " + potenciaMotores[i] + " W");  
    }  
    System.out.println("Motor seleccionado: " + motorSeleccionado);  
    System.out.println("Modo de vuelo: " + modoVuelo);  
    System.out.printf("Velocidad estimada: %.2f km/h\n", velocidadEstimada());  
}  
}
```

3. Hacer un programa en el que se declare un objeto de la clase DroneCarrera, asignándole al objeto los valores:

```
peso = 850  
autonomia = 20  
velocidadMaxima = 100  
numMotores = 4  
potenciaMotores = {120, 125, 118, 122}  
motorSeleccionado = 2  
modoVuelo = "asistido"
```



Visualizar los datos del dron, así como su velocidad estimada.

Modificar los siguientes valores del objeto:

autonomia = 25
modoVuelo = "acrobático"
potenciaMotores = {130, 135, 128, 133}
motorSeleccionado = 3

Volver a mostrar todos los datos y la nueva velocidad.

Ejemplo de la ejecución del programa:

run:

--- Datos del Drone de Carreras ---

Peso: 850 gramos

Autonomía: 20 minutos

Velocidad Máxima: 100 km/h

Número de motores: 4

Potencia de motores:

- Motor 1: 120 W

- Motor 2: 125 W

- Motor 3: 118 W

- Motor 4: 122 W

Motor seleccionado: 2

Modo de vuelo: asistido

Velocidad estimada: 6,18 km/h

--- Datos tras la modificación ---

Peso: 850 gramos

Autonomía: 25 minutos

Velocidad Máxima: 100 km/h

Número de motores: 4

Potencia de motores:

- Motor 1: 130 W

- Motor 2: 135 W

- Motor 3: 128 W

- Motor 4: 133 W

Motor seleccionado: 3

Modo de vuelo: acrobático

Velocidad estimada: 7,89 km/hBUILD SUCCESSFUL (total time: 0 seconds)

Podéis incluir el siguiente código para que visualice acentos y caracteres especiales como la ñ:

```
import java.io.UnsupportedEncodingException;  
import java.util.logging.Level;
```



```
import java.util.logging.Logger;

System.setProperty("file.encoding", "UTF-8");
try {
    System.setOut(new java.io.PrintStream(System.out, true, "UTF-8"));
} catch (UnsupportedEncodingException ex) {
    Logger.getLogger(PruebaDroneCarrera.class.getName()).log(Level.SEVERE, null, ex);
}
```

Solución:

```
package pruebadron;

import dronecarrera.DroneCarrera;
import java.io.UnsupportedEncodingException;
import java.util.logging.Level;
import java.util.logging.Logger;

public class PruebaDroneCarrera {

    public static void main(String[] args) {
        System.setProperty("file.encoding", "UTF-8");
        try {
            System.setOut(new java.io.PrintStream(System.out, true, "UTF-8"));
        } catch (UnsupportedEncodingException ex) {
            Logger.getLogger(PruebaDroneCarrera.class.getName()).log(Level.SEVERE, null, ex);
        }

        int[] potencias1 = {120, 125, 118, 122};
        DroneCarrera dron = new DroneCarrera(850, 20, 100, 4, potencias1, 2, "asistido");

        System.out.println("--- Datos del Drone de Carreras ---");
        dron.visualizaDroneCarrera();

        // Modificaciones
        dron.setAutonomia(25);
        dron.setModoVuelo("acrobático");
        int[] potencias2 = {130, 135, 128, 133};
        dron.setPotenciaMotores(potencias2);
        dron.setMotorSeleccionado(3);

        System.out.println("\n--- Datos tras la modificación ---");
        dron.visualizaDroneCarrera();
    }
}
```

4. Hacer el ejercicio “132 Las cartas del abuelo” de acepta el reto.



Solución:

```
import java.util.Scanner;

public class Ejercicio132 {

    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);
        String texto;
        int casos;
        int n1, n2;
        int cont;
        int[] aux;
        while (true) {
            texto = teclado.nextLine();
            casos = teclado.nextInt();
            if (casos == 0) {
                break;
            }
            cont = 0;
            aux = new int[texto.length()];
            char a = texto.charAt(0);
            for (int i = 1; i < texto.length(); i++) {
                if (texto.charAt(i) != a) {
                    a = texto.charAt(i);
                    cont++;
                }
                aux[i] = cont;
            }
            for (int i = 0; i < casos; i++) {
                n1 = teclado.nextInt();
                n2 = teclado.nextInt();
                if (aux[n1] == aux[n2]) {
                    System.out.println("SI");
                } else {
                    System.out.println("NO");
                }
            }
            teclado.nextLine();
            System.out.println();
        }
    }
}
```

5. Hacer el ejercicio “417 Nana al bebé de papá y mamá” de acepta el reto.

Solución:



```
import java.util.Scanner;

public class NanaAlBebeDePapaYMama {

    public static void main(String[] args) {
        String palabra;
        int longitud, j, pos, i;
        boolean acabe;
        Scanner leo = new Scanner(System.in);

        while (leo.hasNext()) {
            palabra = leo.nextLine();
            palabra = palabra.toLowerCase();

            acabe = false;
            for (i = 1; i <= palabra.length() / 2 && !acabe; i++) {
                for (j = 0, pos = 0; j < palabra.length() / i; j++, pos += i) {
                    if (!palabra.substring(0, i).equals(palabra.substring(pos, pos + i))) {
                        break;
                    }
                }
            }

            if (j == palabra.length() / i && (i == 1 || palabra.length() % i == 0)) {
                acabe = true;
                System.out.println(i);
            }
        }
        if (!acabe) {
            System.out.println(palabra.length());
        }
    }
}
```

Criterios de puntuación.

Ejercicio 1: 2 puntos
Ejercicio 2: 3 puntos
Ejercicio 3: 2 puntos
Ejercicio 4: 1,5 puntos
Ejercicio 5: 1,5 puntos

Indicaciones de entrega.

Enviar un único fichero, en cualquier formato excepto pdf. El documento tiene que incluir el código fuente de todos los ejercicios y una captura de pantalla de los ejercicios de "Acepta



el reto” donde se vea el veredicto del juez automático. El nombre de fichero tiene que estar formado por vuestro nombre y apellidos.